RallyPoint — Milestone 2 演示视频脚本（后端组件 / Back-End Component）

总时长约 7:50 | 英文口播约 1,180 词 | 覆盖评分项：API List + Test Case Documentation /20 · Database Design & Implementation /20 · API Testing Results & Live Demonstration /60 | 本里程碑 = ACIS 498 Milestone 2 后端组件（Flask REST API + SQLAlchemy 数据库）；React 前端属于 Milestone 1 | 全程真实代码、真实接口、真实数据库写入，无夸大

开场 OPENING | 0:00–0:45 | 定调 + 本里程碑范围

【画面/操作】 全屏打开 VS Code，停在 `README.md` 的架构图上，不滚动。

> 「念」 Hi, I'm Phoebe. This is Milestone 2 of RallyPoint, my ACIS 498 capstone — an AI partner-matcher for recreational tennis and pickleball players. Milestone 1 was the React front end; today is the back end: a Flask REST API, fully integrated with a relational database.

【画面/操作】 光标扫过架构图里的 Flask 层与数据库层。

> 「念」 Concretely, that's an application factory with nine blueprints exposing fifty-five endpoints over fifteen database tables, with JWT authentication, Pydantic request validation, and the explainable matching service. I'll cover three things, matching the rubric: the database design, the API surface, and — the core of the grade — the API tests and a live run that proves the database updates after every single operation.

（约 115 词）

数据库设计 DATABASE DESIGN | 0:45–2:30 | 得分项：Database Design & Implementation /20

【画面/操作】 打开 `docs/MILESTONE_2_DATABASE_DESIGN.md`，停在 ER 图。

> 「念」 Let's start with the data model. The schema is fifteen tables. This is the entity-relationship diagram — users at the center, with everything else hanging off it through real foreign keys.

【画面/操作】 打开 `backend/models/` 文件夹，点开 `user.py`，指向 `User` 与 `SportProfile`。

> 「念」 The first design decision: profiles are per-sport, not per-user. Skill, availability, and home court live in a separate `sport_profiles` table — one row per sport — because someone can be a four-oh tennis player and a three-oh pickleball player. A unique constraint on user plus sport enforces that.

【画面/操作】 点开 `session_model.py`，指向 `host_id` / `guest_id` / `status`。

> 「念」 A session links two users — a host and a guest — with two directed foreign keys. Its status is viewer-relative: the same open invite reads as "requested" to the guest and "pending" to the host, so the API serves each person the right view of one row.

【画面/操作】 点开 `game_invite.py`，指向 `phase` 与 `session_id`；再点 `ai_match_log.py`，指向 `UniqueConstraint(viewer_id, candidate_id, sport)`。

> 「念」 Two more I want to call out. Game invites are two-phase — first you agree to play, then you settle the time — and when a time locks, the invite materializes a real session row and points at it by `session_id`, so the calendar only ever reads one table. And `ai_match_logs` has a unique constraint on viewer, candidate, and sport, so the match-reason endpoint upserts instead of appending — the verdict stays stable across page loads.

【画面/操作】 快速划过 `docs/schema_ddl.sql`（真实生成的建表语句）。

> 「念」 And this is the actual generated DDL — the create-table statements the models produce. The same SQLAlchemy code runs on SQLite in development and PostgreSQL in production; switching is one environment variable, no code change, and it's already live on Render.

（约 285 词）

API 接口与文档 API SURFACE ｜ 2:30–4:15 ｜ 得分项：API List + Test Case Documentation /20

【画面/操作】 终端运行 `python app.py`，浏览器打开 `http://localhost:5050/api/docs/`（Swagger UI）。

> 「念」 Now the API. Instead of a static list, I integrated Swagger — OpenAPI — so the whole surface is self-documented and testable from the browser. These are the endpoints, grouped by area: auth, players, sessions, courts, appointments, invites, admin, support.

【画面/操作】 展开 `POST /auth/login`，点 "Try it out"，填入 `{"email":"alex@rally.app","password":"rally1234"}`，Execute，复制返回的 token。

> 「念」 Let me actually call it. `POST /auth/login` with a seeded account. It returns the user and a JWT, signed with HS256 and valid for seven days. For browsers that token rides in an httpOnly cookie the JavaScript never touches, with a CSRF token on writes; for API clients like this, I pass it as a Bearer header.

【画面/操作】 点 "Authorize"，粘贴 `Bearer <token>`；展开 `GET /players?sport=Pickleball`，Execute。

> 「念」 I authorize with that token, then hit `GET /players`. Here's the matching output: each candidate comes back with a match score zero to a hundred, a tier, and reason chips. The score is a transparent heuristic — skill closeness up to forty-five points, court proximity up to twenty-five, weekly schedule overlap up to twenty, a shared home court up to fifteen, same sport plus five — and every point maps to a chip, so it's auditable, not a black box.

【画面/操作】 展开 `POST /auth/signup`，填入 `{"email":"not-an-email","password":"short","name":""}`，Execute，展示 422。

> 「念」 And every write body is validated by a Pydantic schema. Send bad input — a malformed email, a weak password, a blank name — and the API returns a structured four-twenty-two with per-field errors, never a five-hundred crash. My API reference document lists all fifty-five endpoints this way, each with its expected input and a sample JSON input and output.

(约 300 词)

现场演示：每次操作的数据库写入 LIVE DB EVIDENCE ｜ 4:15–6:45 ｜ 得分项：API Testing Results & Live Demonstration /60

【画面/操作】 切到终端，运行 `python api_demo.py | less`（或直接滚动 `docs/api_demo_output.txt`）。

> 「念」 This is the heart of the submission — the part the assignment asks for directly: proof that the database updates after each operation. I wrote a harness, `api_demo.py`. It builds a fresh database, then makes thirty-six real API calls, and after every one it prints three things: the request, the HTTP response, and the exact change in the database.

【画面/操作】 停在 STEP 1（signup），指向 `DB DELTA: users 2->3 (+1); sport_profiles 2->3 (+1)`。

> 「念」 Here's signup. The response is a two-oh-one, and the database delta shows it: the users table went from two rows to three, and sport_profiles from two to three. One insert into each — exactly what a new account should write.

【画面/操作】 滚到 STEP 15（accept session），指向 `ROW BEFORE ... status='pending'` 与 `ROW AFTER ... status='confirmed'`.

> 「念」 Now an update, which doesn't change the row count — so the harness shows the row itself, before and after. The guest accepts the request, and you can see this session row flip from status "pending" to "confirmed." That's a real state transition, captured live.

【画面/操作】 滚到 STEP 22–25（open game join/waitlist/leave/promote），依次指出 participant 的 `+1`、waitlisted、`-1`、promote。

> 「念」 Here's the open-game waitlist, four steps in a row. Someone creates a game capped at two, a second player joins, a third joins — and because it's full, they're added as waitlisted. Then a confirmed player leaves, the participant row is deleted, and the waitlisted player is automatically promoted into the open spot. You can watch the participant rows move.

【画面/操作】 滚到 STEP 30（accept invite time），指向 `sessions 1->2 (+1)` 与 invite 行 `phase -> confirmed, session_id -> 2`。

> 「念」 And here's the invite materialization I mentioned. The non-proposer accepts the time, a brand-new confirmed session is inserted, and the invite row updates in place — phase to "confirmed," and session_id now points at that new session. Two tables changing together, in one operation.

【画面/操作】 滚到 STEP 35（admin review report + suspend），指向 report `status open->reviewed` 与 user `is_active 1->0`。

> 「念」 One more — the admin side. An admin resolves a trust-and-safety report and, in the same call, suspends the reported account: the report status goes from "open" to "reviewed," and that user's `is_active` flag flips from one to zero. Inserts show a plus-one, deletes a minus-one, and in-place updates show the fields before and after. Every write is accounted for.

(约 375 词)

测试与 CI TESTS & CI ｜ 6:45–7:25 ｜ 得分项：API Testing Results（自动化测试）

【画面/操作】 新终端运行 `pytest -v`，停在绿色的 `120 passed`。

> 「念」 Beyond that live run, the back end has an automated suite: a hundred and twenty pytest cases across fourteen files, all passing. They cover the whole surface — auth, matching, sessions, invites, appointments, courts, admin, support — plus the security invariants, like a test that a spoofed user-id header can't bypass the JWT.

【画面/操作】 打开 `.github/workflows/ci.yml`，指向测试步骤；再快速划过 `docs/MILESTONE_2_TEST_RESULTS.md` 的测试用例表。

> 「念」 Each test runs against a fresh in-memory database, so they're fully isolated, and they run on every push through GitHub Actions. My test-results document pairs this with a case table — input, expected result, status — and the full per-operation transcript you just saw.

(约 130 词)

结尾 CLOSING ｜ 7:25–7:50 ｜ 收束 + 路线图

【画面/操作】 回到 Swagger UI 的接口总览页。

> 「念」 So that's Milestone 2: fifty-five endpoints over fifteen tables, JWT auth, Pydantic validation, an explainable matching service — all integrated with the database, and verified two ways: a hundred and twenty automated tests, and a live run proving the write after every operation. The clear next step is wiring `ai_match_logs` on the served path into a learning-to-rank loop, once there's real accept-decline history. Thanks for watching.

(约 80 词)

英文口播合计约 1,185 词（115 + 285 + 300 + 375 + 130 + 80）；按 ~150 词/分钟实读约 7:50，控制在 8:30 以内并留有余量。若现场偏长，优先压缩 API Surface 段的 Pydantic 演示。

录制小贴士

- 录前先跑一遍环境：`cd backend && source .venv/bin/activate && pytest -v`（看到 `120 passed`）、`python api_demo.py`（确认能打印 DB DELTA）、`python app.py` 打开 `/api/docs/`。三样都正常再开录，避免现场卡壳。
- 60 分在"现场演示每次操作后的数据库写入"这一段——这是整个视频的重心，务必讲透。 建议直接滚 `docs/api_demo_output.txt`（字号调大、深色主题），或 `python api_demo.py | less` 现场跑。每种写入各挑一个例子讲清楚："insert 看 `+1` 行、delete 看 `-1` 行、原地 update 看 `ROW BEFORE / ROW AFTER` 的字段前后对比"。评委是 Microsoft 首席 AI 架构师，看的是"你真的证明了 DB 被更新"，不是口头声称。
- Database Design 段按 ER 图 → `user.py`（per-sport 设计）→ `session_model.py`（两个外键 + viewer-relative status）→ `game_invite.py` + `ai_match_log.py`（materialize + upsert 唯一约束）→

`schema_ddl.sql` 的顺序走，每张表停一拍。强调 SQLite→Postgres 一个 env var 切换、已上线 Render。

- API Surface 段以 Swagger `/api/docs/` 为中心：login 拿 token → Authorize → `GET /players` 看真实分数与 reason chips → `POST /auth/signup` 故意传坏数据看 422。认证务必说 httpOnly cookie + CSRF（浏览器）/ Bearer header（API 客户端），不要说 localStorage。 匹配算法念分值时放慢：skill 0–45、proximity 0–25、schedule 0–20、home court 0–15、same sport +5。
- 诚实边界：`ai_match_logs` 在"服务路径"上尚未落地日志（只有 `/api/ai/match-reason` 按需写），结尾把它讲成"下一步"，别声称已在 learning-to-rank；无 `OPENAI_API_KEY` 时 embedding 信号贡献为 0、LLM 关闭——如被问到就说"graceful degradation，分数纯由透明信号得出"。
- 编辑器/终端字号 16–18pt、深色主题、开行号；标签页提前排好零延迟切换；念到 "the database delta shows it""before and after""materializes a real session""is_active flips from one to zero""120 passed" 时各停一拍。
- 全程只展示真实代码、真实接口调用、真实数据库写入；最后一句后停 1 秒再停录。